

《计算概论》课程 程序设计部分

C/C++语言基本成分 —— 数据成分

董 豪

北京大学 计算机学院 前沿计算研究中心

hao.dong@pku.edu.cn

北京大学



再谈，我们的进度安排



北京大学



程序设计语言的构成

- **语言的种类千差万别，但是，一般说来，基本成分不外四种：**
  - **数据成分**，用以描述程序中所涉及的数据；
  - **运算成分**，用以描述程序中所包含的运算；
  - **控制成分**，用以表达程序中的控制构造；
  - **传输成分**，用以表达程序中数据的传输；

——计算机科学技术百科全书

北京大学

问题1：

是不是 无论我们在程序里写什么“单词”，计算机都能明白？

问题2：

是不是 无论我们在程序里写什么“数”和“计算符号”，计算机都能明白？

问题3：

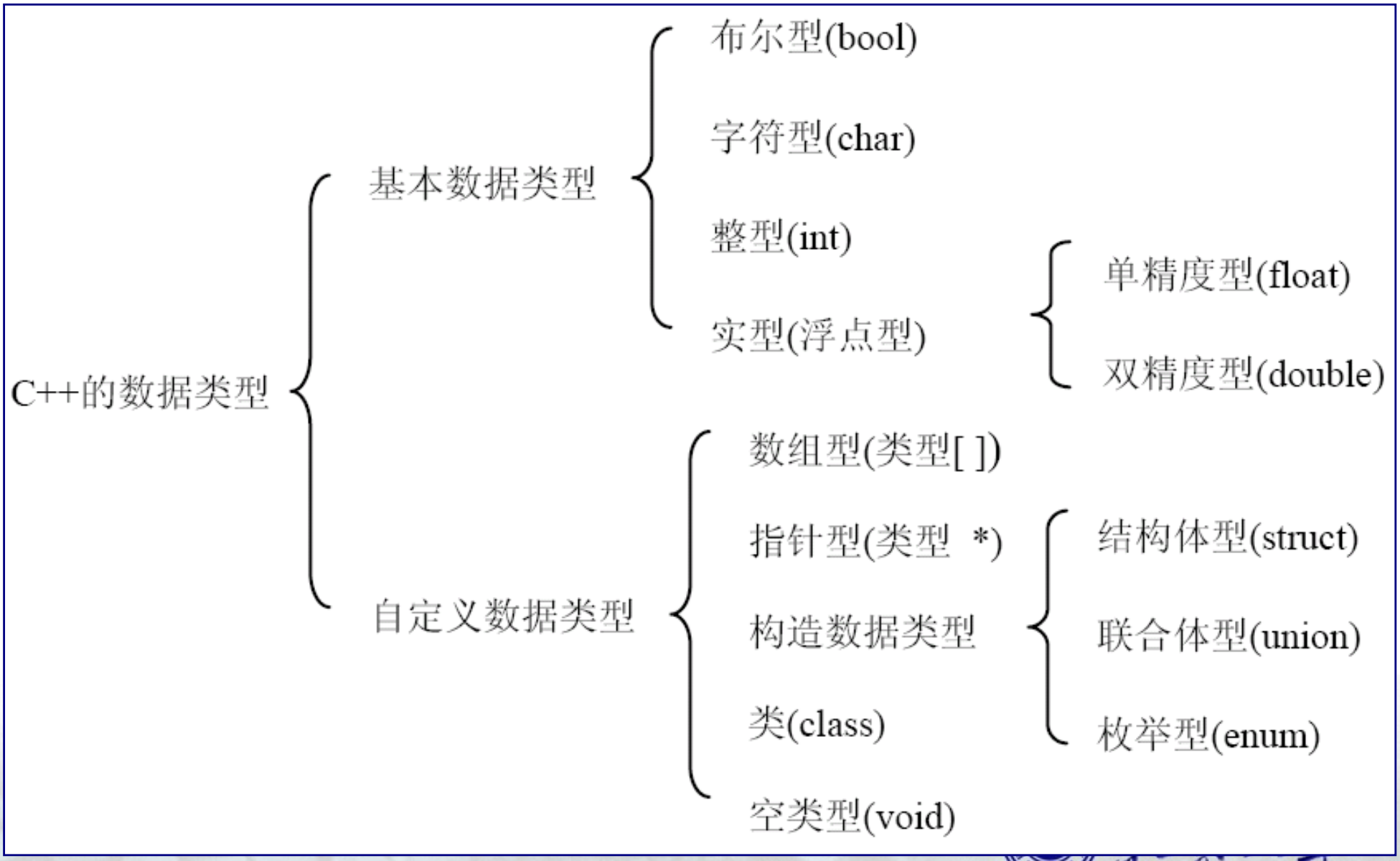
世界上可能要用“程序来表达的逻辑”纷繁复杂，程序设计语言里面得有多少种“句式”才能够用？

问题2：

是不是 无论我们在程序里写什么“数”和“计算符号”，计算机都能明白?

- **NO!**
- 计算机只能“看懂”某些类型的数据，这些**“数据的类型”**和相应的“操作符号”也是定义好的。

计算机能够看懂的数据的类型





程序设计语言的构成

- **语言的种类千差万别，但是，一般说来，基本成分不外四种：**
  - **数据成分**，用以描述程序中所涉及的数据；
  - **运算成分**，用以描述程序中所包含的运算；
  - **控制成分**，用以表达程序中的控制构造；
  - **传输成分**，用以表达程序中数据的传输；

——计算机科学技术百科全书

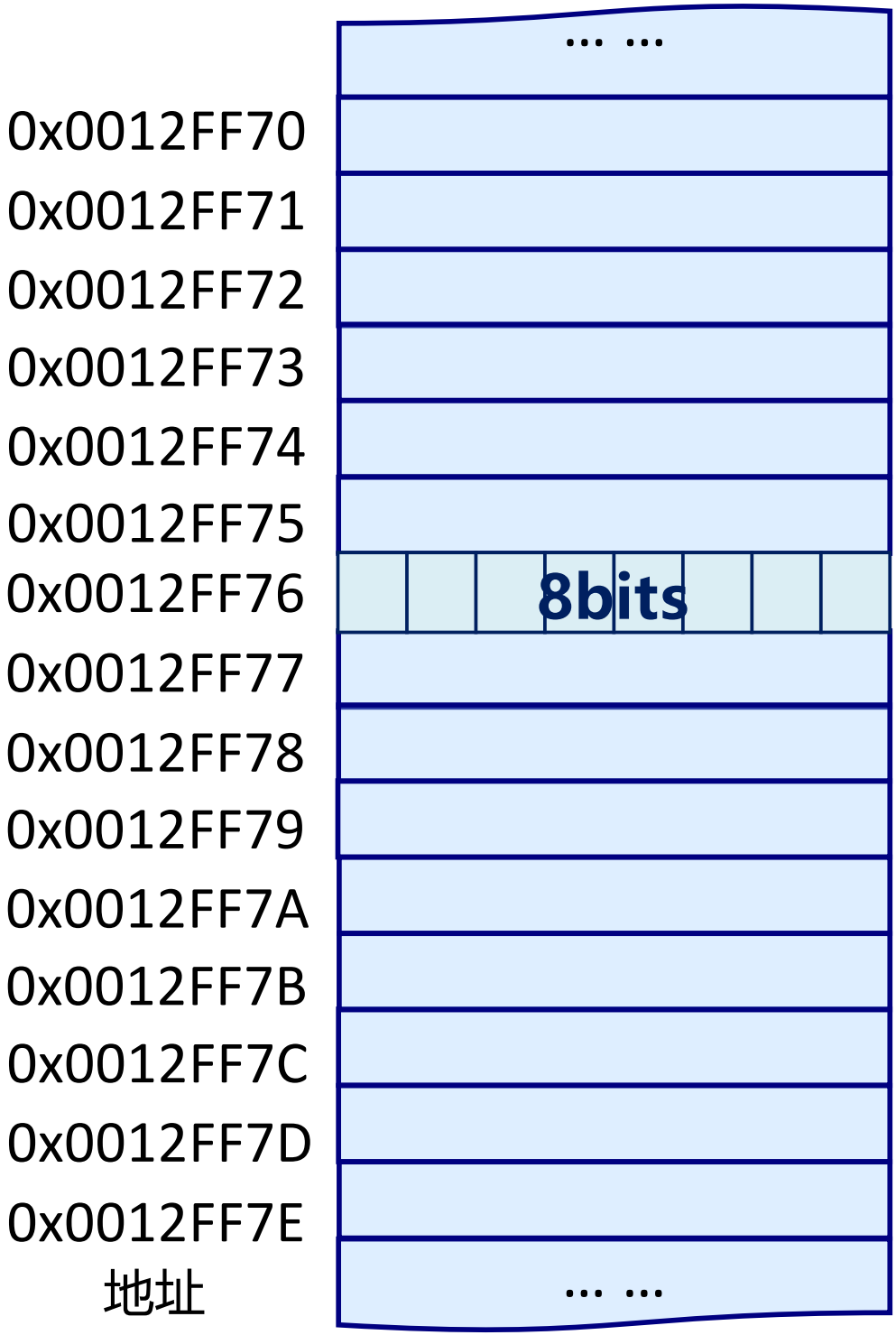


C/C++ 程序设计语言的基本构成

—— 数据成分



北京大学



■ 存储空间单位

2^{10}=1024

1B (Byte) = 8 b (bit)

1KB = 1024Byte

1MB = 1024KB

1GB= 1024MB

1TB = 1024GB

1PB = 1024TB

```cpp
#include <iostream>
using namespace std;
int main() {
    int number[45] = {78, 56, 69, 31, 36, 67, 31, 47,
                      69, 34, 45, 74, 61, 82, 43, 41,
                      76, 79, 81, 66, 54, 50, 76, 51,
                      53, 28, 74, 39, 45, 61, 52, 41,
                      43, 75, 78, 84, 72, 51, 43, 64,
                      75, 81, 69, 55, 74};
    int max = 0;
    int i = 0;
    for (i = 0; i < 45; i++) {
        if (number[i] > max) max = number[i];
    }
    cout << "The Maximal Number is:" << max;
    return 0;
}
```

数据成分



变量：先定义 再使用

- “值可以变化的量”
- 变量的定义格式

类型说明符 变量名;

类型说明符 变量名 = *初始化值*;

```
int max; //合法但没初始化
int max = 0;
char character;
char character = 'A';
double result = 12.345;
```



C++ 的97个关键字

alignas	constinit	long	struct
alignof	const_cast	mutable	switch
and	continue	namespace	synchronized
and_eq	co_await	new	template
asm	co_return	noexcept	this
atomic_cancel	co_yield	not	thread_local
atomic_commit	decltype	not_eq	throw
atomic_noexcept	default	nullptr	true
auto	delete	operator	try
bitand	do	or	typedef
bitor	double	or_eq	typeid
bool	dynamic_cast	private	typename
break	else	protected	union
case	enum	public	unsigned
catch	explicit	reflexpr	using
char	export	register	virtual
char8_t	extern	reinterpret_cast	void
char16_t	false	requires	volatile
char32_t	float	return	wchar_t
class	for	short	while
compl	friend	signed	xor
concept	goto	sizeof	xor_eq
const	if	static	
consteval	inline	static_assert	
constexpr	int	static_cast	

关于C/C++程序的标识符

■ 什么是标识符

- ◆用来标识对象名、引用、函数名、类名、goto 标号，以及其他实体的有效字符序列称为标识符(identifier)。
- ◆**C++语言规定：**标识符只能由字母、数字和下划线三种字符组成，且**第一个字符必须为字母或下划线**，且**不可与关键字或保留字相同。**
- ◆但请尽量不要用下划线开头。（被保留）

■ **合法的标识符：**

```
sum, average, total, day, month,
Student_name, lotus_1_2_3, basic, li_ling
```

■ **不合法的标识符：**

```
M.D.John, ￥123, #33, 3D64, a>b
```

```cpp
#include<iostream>
#include<cmath>
using namespace std;

int main()
{
    cout << max(3,4) << endl;
    return 0;
}
```

```
4
```

```cpp
#include<iostream>
#include<cmath>
using namespace std;

int main()
{
    cout << max(3,4) << endl;
    int max = 100;
    cout << max
    return 0;
}
```

```
4
100
```

```cpp
#include<iostream>
#include<cmath>
using namespace std;

int main()
{
    cout << max(3,4) << endl;
    int max=100;
    cout << max << endl;
    cout << max(5,6) << endl;
    return 0;
}
```

```
main.cpp: In function 'int main()':
main.cpp:13:20: error: 'max' cannot be used as a functio
   13 |     cout << max(5,6) << endl;
      |                    ^


** Process exited - Return Code: 1 **
```



```
int mm = 0, nn = 0, kk=0;
....
```

```
int zyczbt = 0;
...
```

```
int i1 = 0, i2 = 0, i3 = 0;
...
```

```
int haha = 0;
char mingzi = ' ';
```

```
int int1 = 1;
...
```



关于变量的名字



关于变量的命名：驼峰命名法

- 驼峰命名法（骆驼式命名法）
  - 由一个或多个单词连结在一起；
  - 第一个单词以小写字母开始；
  - 第二个单词的首字母大写或每一个单词的首字母都采用大写字母；

myFirstName, myLastName, nextStudentName,

intCount, printEmployeePaychecks()

关于变量的命名：首字母大小写问题

```
#include<iostream>
using namespace std;

class Student{
    public:
        char name[20] = "default";
        int age = 18;
        int studentID = 100;
};

int main()
{
    Student stu1;
    cout << stu1.name << ' ' << stu1.age << ' ' << stu1.studentID << endl;
    return 0;
}
```

- 变量名第一个字母通常小写
- 类名第一个字母通常大些

```
default 18 100
```

关于变量的命名：匈牙利命名法



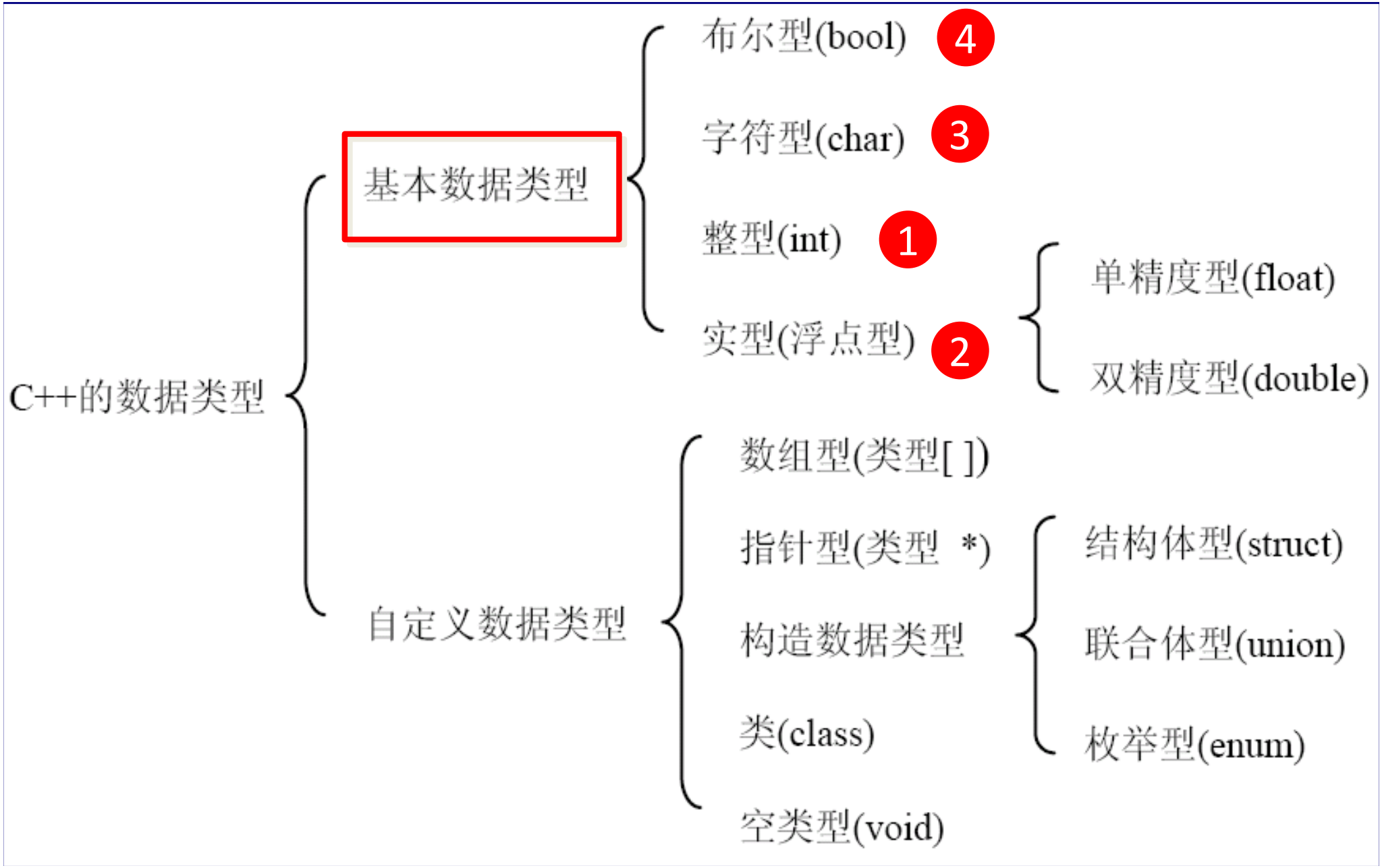
- 匈牙利命名法

  由Microsoft著名开发人员Excel主要设计者Charles Simonyi在其博士论文中提出。

1. 以一个或者多个小写字母开头，来指定数据类型。
2. 其后是一个或者多个第一个字母大写的单词，指出变量的用途。

如： chGrade; nLength; bOnOff; strStudentName;

C/C++ 程序中的数据类型


C++的数据类型
基本数据类型
布尔型(bool)
4
字符型(char)
3
整型(int)
1
实型(浮点型)
2
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)

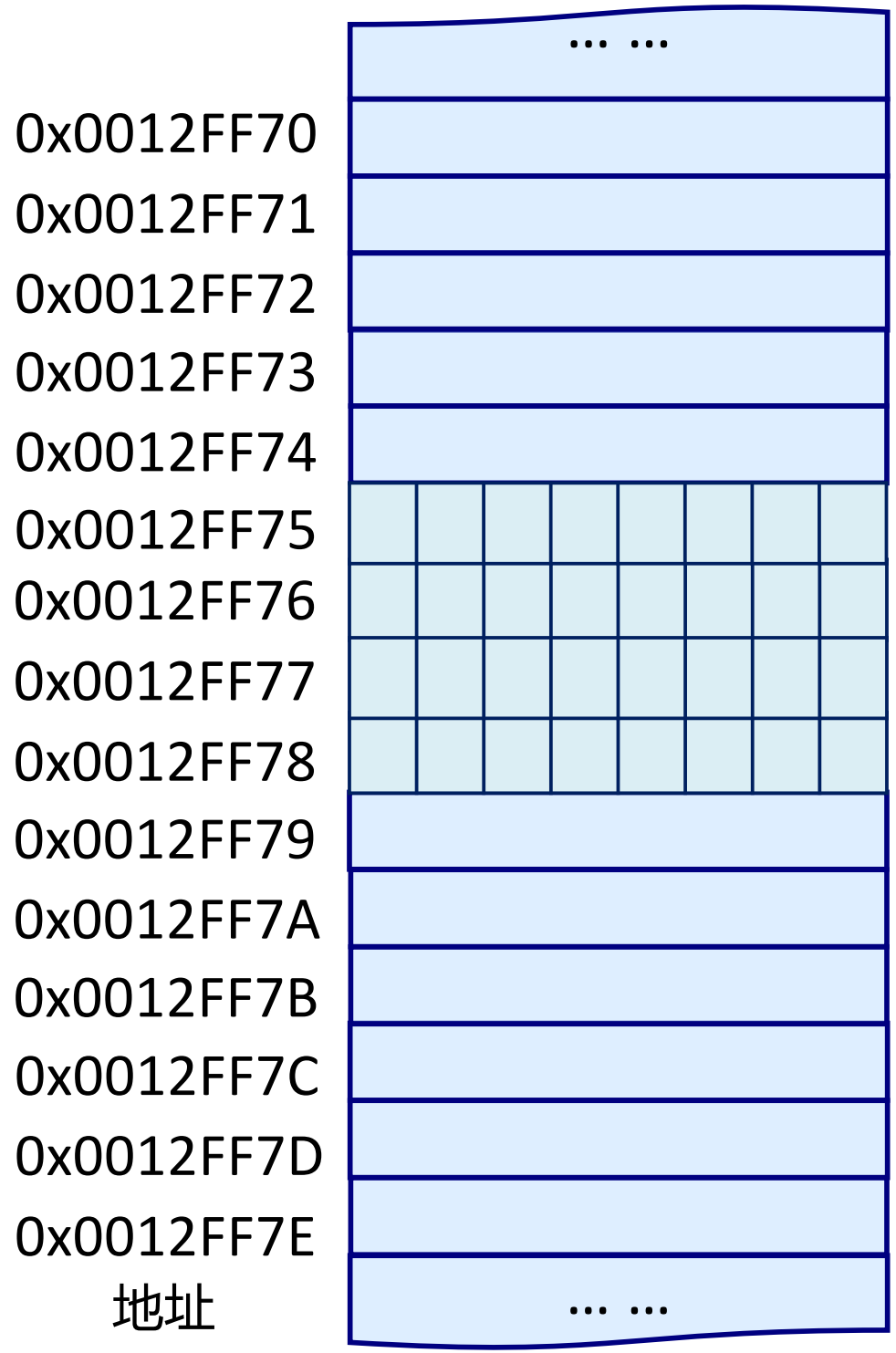


整型数据

```
int a = 0;
```

请这样来 想象 内存

```
int  a = 0;
```

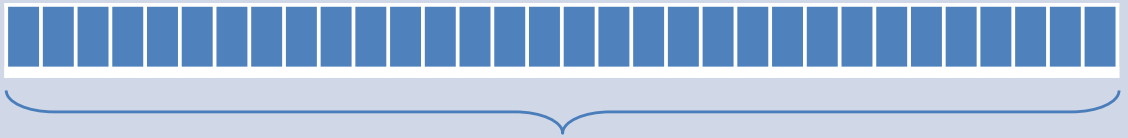




整型数据的分类

分类	类型说明符
基本型	int
短整型	short short int
长整型	long long int
扩展长整型	long long long long int

整型数据的分类

	整型	内存空间
基本型	**int**	 一般32bit
短整型	**short** short int	 一般16bit
长整型	**long** long int	 一般32bit
扩展长整型	long long long long int	 一般64bit

【注】C 标准没有具体规定以上各类数据所占内存字节数，只要求long型数据长度不短于int型，short型不长于int型。

变量的初始化

```cpp
#include <iostream>
using namespace std;
int main() {
    int a;
    cout << a << endl;
    return 0;
}
```

```
15740
Program ended with exit code: 0
```

Visual Studio (MSVC) :

```
number is: 4201998
```

变量的初始化

```
#include<iostream>
using namespace std;
int main() {
    int a, b, c;
    cout<<a<<endl;
    cout<<b<<endl;
    cout<<c<<endl;
    return 0;
}
```




```
15692
1
65536
Program ended with exit code: 0
```

整型数据的范围

- 一般每种类型所占内存空间和表示的范围：

类型说明符	类型	字节数	[signed]表示范围	unsigned表示范围
int	整型	4	$-2^{31} \sim 2^{31}-1$	$0 \sim 2^{32}$
short [int]	短整型	2	$-2^{15} \sim 2^{15}-1$	$0 \sim 2^{16}$
long [int]	长整型	4	$-2^{31} \sim 2^{31}-1$	$0 \sim 2^{32}$
long long [int]	扩展长整型	8	$-2^{63} \sim 2^{63}-1$	$0 \sim 2^{64}$

整型数据的范围

■ 熟悉一下常用整数的表示范围：

short [int]	2	−32 768～32 767
signed short [int]	2	−32 768～32 767
unsigned short [int]	2	0～65 535
int	4	−2 147 483 648～2 147 483 647
signed int	4	−2 147 483 648～2 147 483 647
unsigned int	4	0～4 294 967 295
long [int]	4	−2 147 483 648～2 147 483 647
signed long [int]	4	−2 147 483 648～2 147 483 647
unsigned long [int]	4	0～4 294 967 295



浮点型

浮点型 = 实型

类型说明符	浮点型分类	长度	有效位	范围
float	单精度浮点型	32位	7位	$-3.4*10^{38}$～$3.4*10^{38}$ 6~7位有效数字
double	双精度浮点型	64位	15位	$-1.7*10^{308}$～$1.7*10^{308}$ 15~16位有效数字
long double	扩展双精度浮点型	不少于double	不少于double	——

有效位：从第一个非零数字起，直到末尾数字止的所有数字的数量

如何知道某种类型的数占多少字节?

■ sizeof 运算符

◆用于计算某种类型的对象在内存中所占的字节数。

```
#include <iostream>
using namespace std;
int main()
{
    cout << "sizeof(short int)=" << sizeof(short int) << endl;
    cout << "sizeof(int)=" << sizeof(int) << endl;
    cout << "sizeof(long int)=" << sizeof(long int) << endl;
    return 0;
}
```

```
sizeof(short int)=2
sizeof(int)=4
sizeof(long int)=8
```

Visual C++时，long为4



整型数据的分类

	有符号	无符号
基本型	int signed int	unsigned int
短整型	short short int signed short signed short int	unsigned short unsigned short int
长整型	long long int signed long signed long int	unsigned long unsigned long int

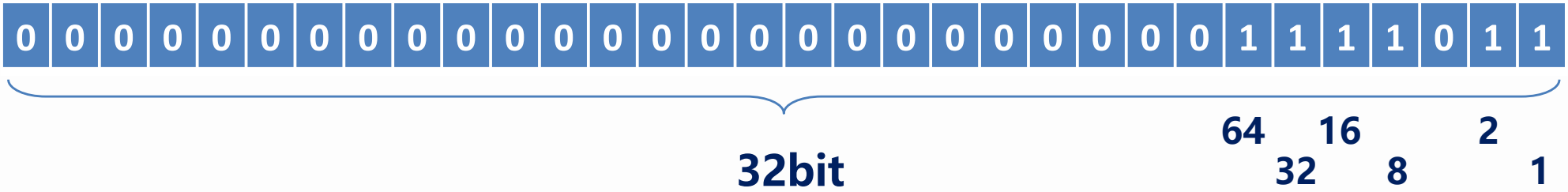
signed vs. unsigned

```
sizeof(short int)=2
sizeof(signed short int)=2
sizeof(int)=4
sizeof(signed int)=4
sizeof(long int)=8
sizeof(signed long int)=8
```

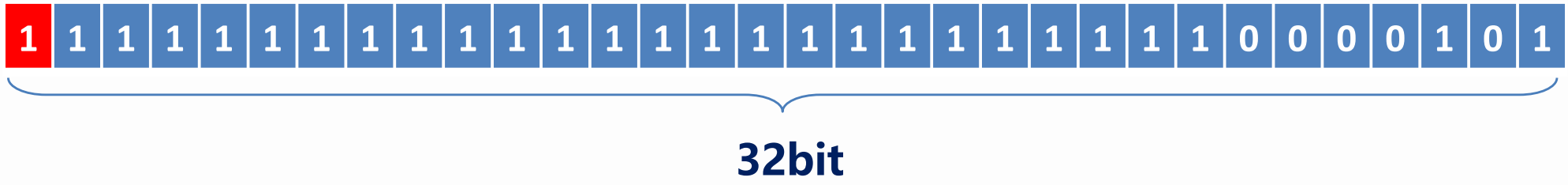
```
#include <iostream>
using namespace std;
int main()
{
    cout << "sizeof(short int)=" << sizeof(short int) << endl;
    cout << "sizeof(signed short int)=" << sizeof(signed short int) << endl;
    cout << "sizeof(int)=" << sizeof(int) << endl;
    cout << "sizeof(signed int)=" << sizeof(signed int) << endl;
    cout << "sizeof(long int)=" << sizeof(long int) << endl;
    cout << "sizeof(signed long int)=" << sizeof(signed long int) << endl;
    return 0;
}
```

signed vs. unsigned

unsigned int i = 123;



signed int i = -123;



signed vs. unsigned

unsigned int i = 123;　　正整数时：补码 = 原码

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 1 1

32bit　取反 … 1 1 0 0 0 0 1 0 0

加一 … 1 1 0 0 0 0 1 0 1

signed int i = -123;

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 1 0 1

32bit

负整数时：原码 ⟷ 取反+1 ⟷ 补码



练习

123 -> 000...0001111011
-123 -> 111...1110000100 +1
-> 111...1110000101

33
-33

77
-77



为什么用补码

正数负数都是递增关系，计算机实现一样

二进制存储	实际表示的数	二进制存储	实际表示的数
0000...0000	0	1000...0000	$-2^{31} = -2147483648$
0000...0001	1	1000...0001	$-2^{31} + 1 = -2147483647$
0000...0010	2	1000...0010	$-2^{31} + 2 = -2147483646$
0000...0011	3	1000...0011	$-2^{31} + 3 = -2147483645$
0000...0100	4	1000...0100	$-2^{31} + 4 = -2147483644$
0000...0101	5	1000...0101	$-2^{31} + 5 = -2147483643$
0000...0110	6	1000...0110	$-2^{31} + 6 = -2147483642$

https://learn-cpp.tk/ch02/part1/type_system/integer_type.html



思考

补码 = 原码取反加一（当负数的时候）
那么补码转原码呢?

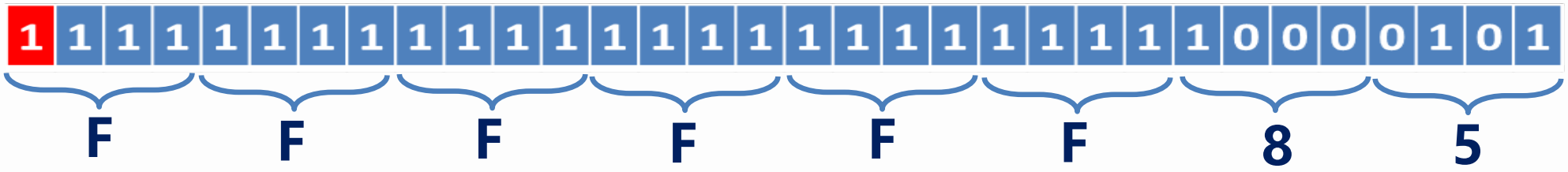
```
 123   -> 000...0001111011
-123   -> 111...1110000100 +1
       -> 111...1110000101

       -> 000...0001111010 + 1
       -> 000...0001111011        -> 123
```

也是取反加一

如何打印一个数的二进制表示?

signed int i = -123;





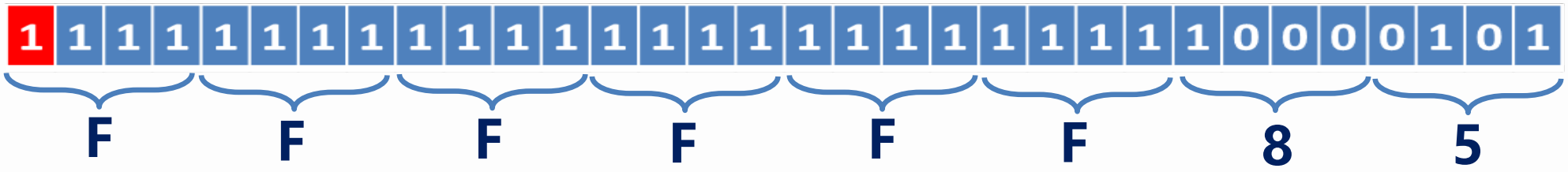
开始之前

二进制	binary
八进制	octonary
十进制	decimal
十六进制	hexadecimal

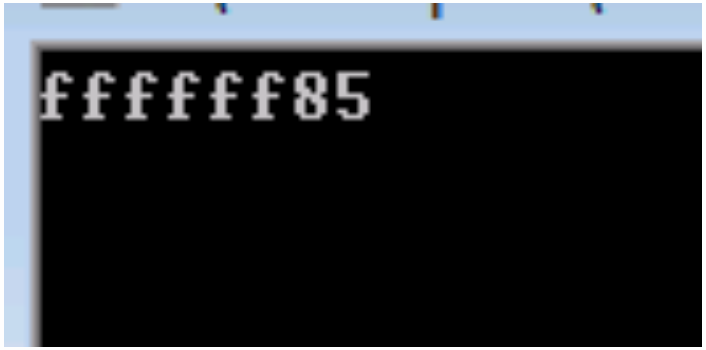
```
00000000000000000000000001111011
00000000000000000000000001111011
00000000000000000000000001111011
using namespace std;
int main()
{
        int a = 123;
        cout << bitset<32>(a) << endl;
        cout << bitset<32>(a / 2) << endl;
        cout << bitset<32>(a >> 1) << endl;
        return 0;
}
```

打印一个数的十六进制表示

signed int i = -123;



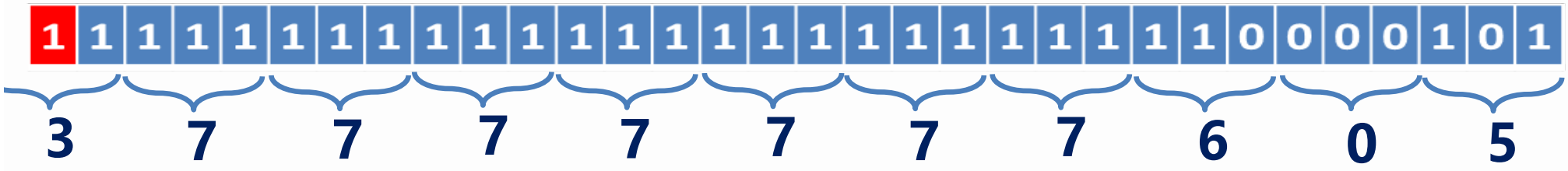

```
#include <iostream>
using namespace std;
int main()
{
        int a = -123;
        cout << hex << a << endl;
        return 0;
}
```



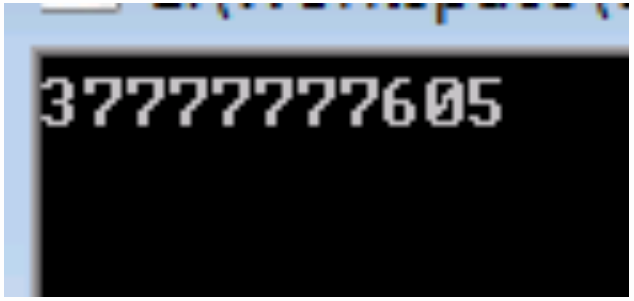
十六进制 hexadecimal

打印一个数的八进制

signed int i = -123;




```
#include <iostream>
using namespace std;
int main()
{
        int a = -123;
        cout << oct << a << endl;
        return 0;
}
```



八进制 octonary

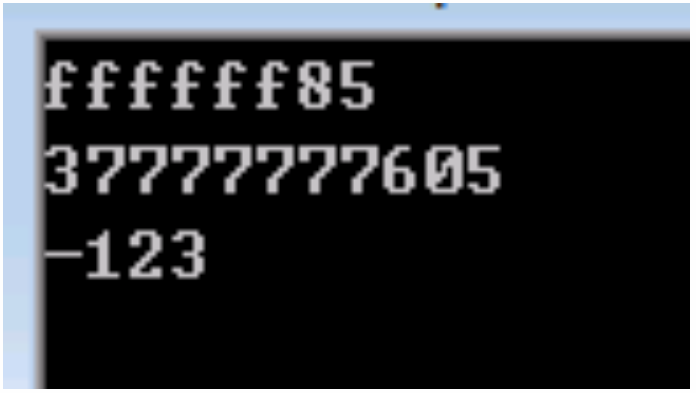
控制符

```
#include <iostream>
using namespace std;
int main()
{
        int a = -123，  b = -124;
        cout << oct << a << endl;
        cout << b << endl;
        return 0;
}
```

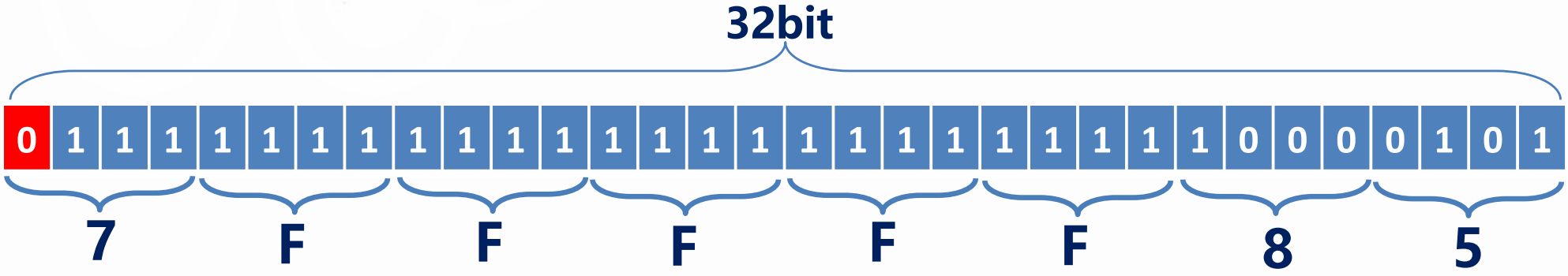
```
37777777605
37777777604
```

以不同的方式输出整数

```
#include <iostream>
using namespace std;
int main()
{
        int a = -123;
        cout << hex << a << endl;
        cout << oct << a << endl;
        cout << dec << a << endl;
        return 0;
}
```

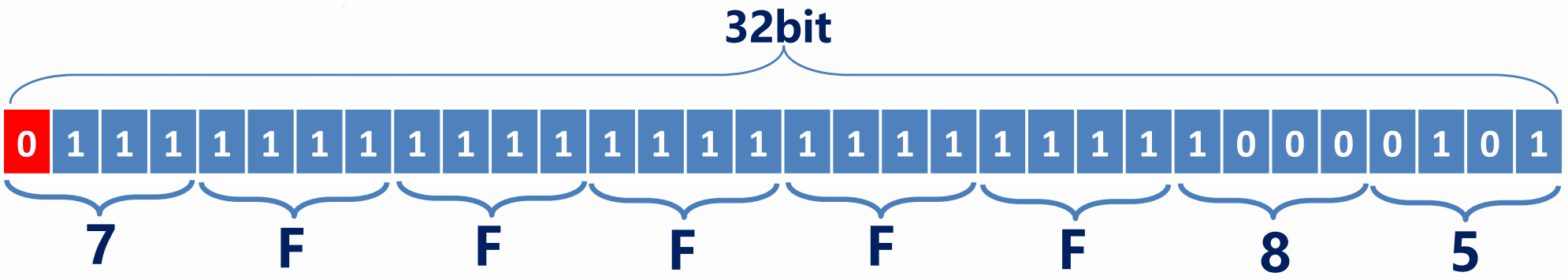


如何把一个二进制数输入程序?



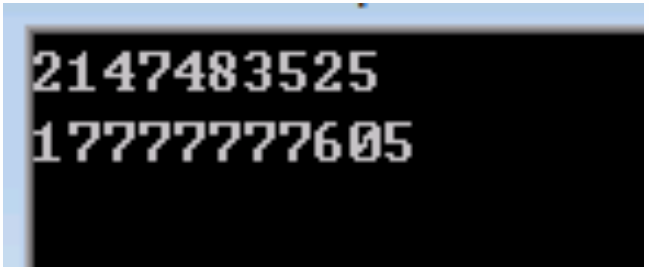
真的0 1 0 1一个个输入电脑？？

把一个十六进制数输入程序



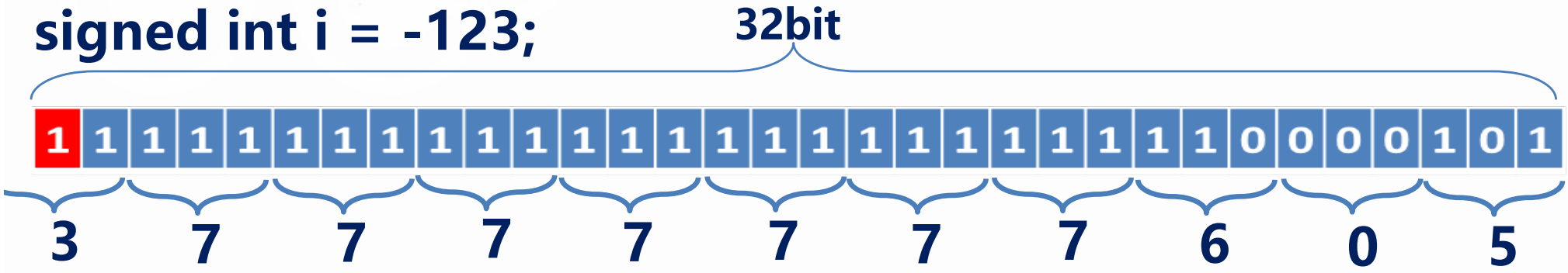

```
#include <iostream>
using namespace std;
int main()
{
        int a = 0x7FFFFF85; // 或者 0x7fffff85 也可
        cout << dec << a << endl;
        cout << oct << a << endl;
        return 0;
}
```

零X 不是 OX

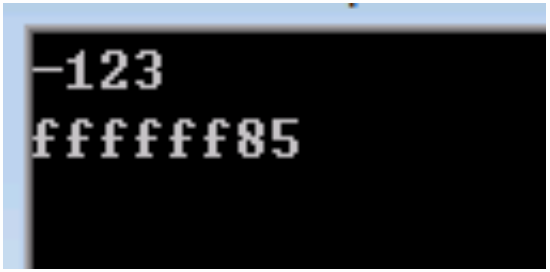


把一个八进制数输入程序

signed int i = -123;

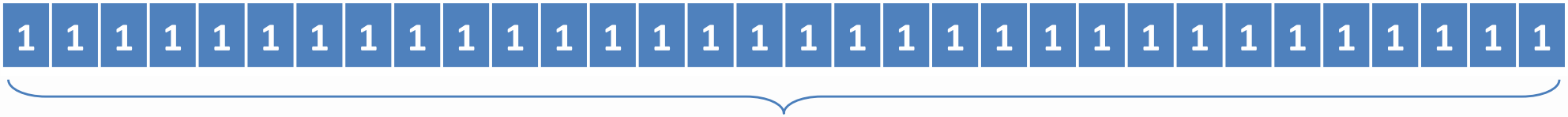



```
#include <iostream>
using namespace std;
int main()
{
        int a = 037777777605;
        cout << dec << a << endl;
        cout << hex << a << endl;
        return 0;
}
```

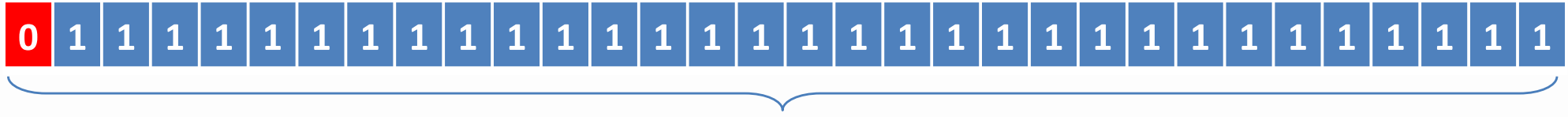


Max & Min

unsigned int i = 0xFFFFFFFF;

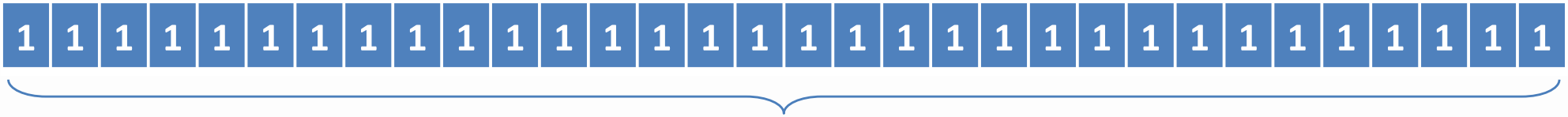


signed int i = 0x7FFFFFFF;



Max & Min

unsigned int i = 0xFFFFFFFF;



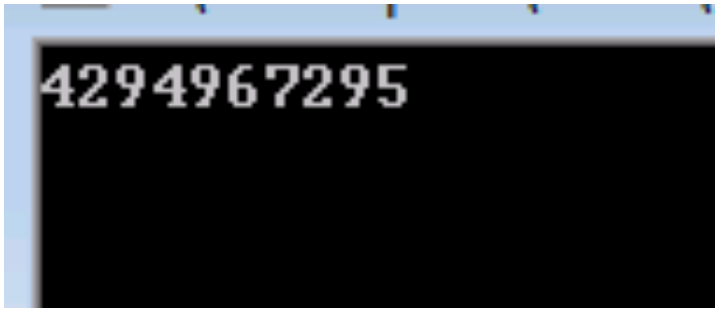
32bit

si

1

1

```
#include <iostream>
using namespace std;
int main()
{
        unsigned int a = 0xFFFFFFFF;
        cout << dec << a << endl;
        return 0;
}
```



2^32 - 1



Max & Min

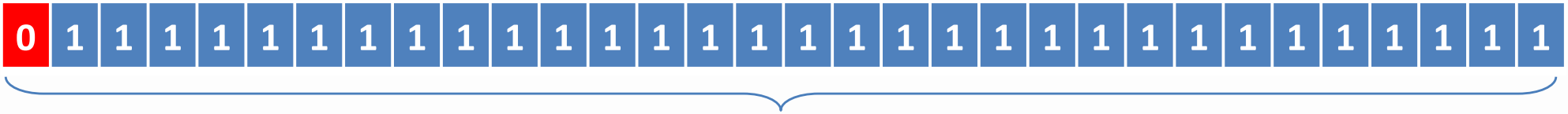
unsigned int i;



unsigned int i; 范围是 0 ~ 2^32 - 1

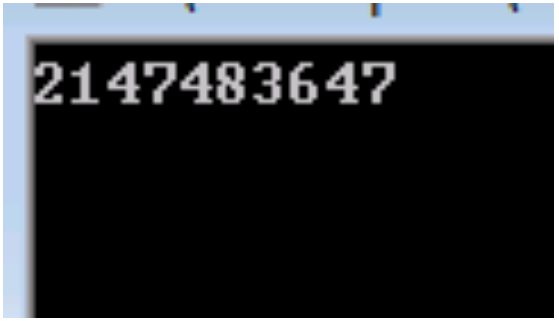
Max & Min

signed int i = 0x7FFFFFFF;



32bit

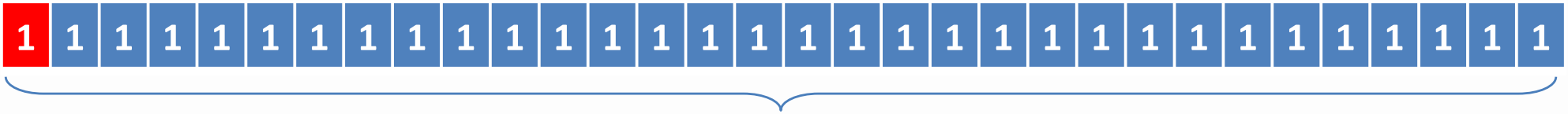
```
#include <iostream>
using namespace std;
int main()
{
        signed int a = 0x7FFFFFFF;
        cout << dec << a << endl;
        return 0;
}
```



2^31 - 1

Max & Min

signed int i = Min;



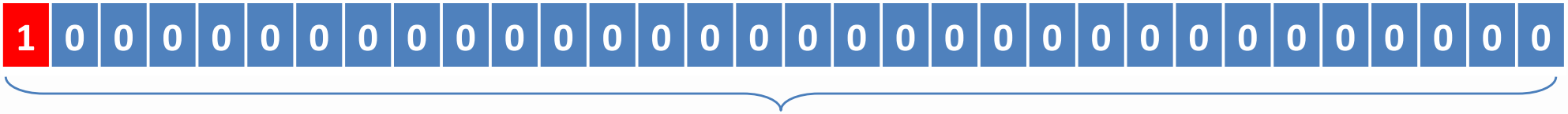
这是有符号数最小的数吗？No!

```
#include <iostream>
using namespace std;
int main()
{
        signed int a = 0xFFFFFFFF;
        cout << dec << a << endl;
        return 0;
}
```

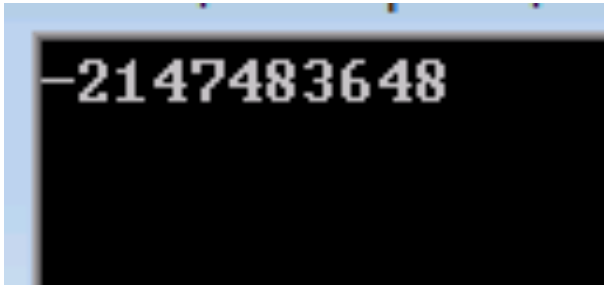
```
-1
请按任意键继续. . .
```

Max & Min

signed int i = Min;

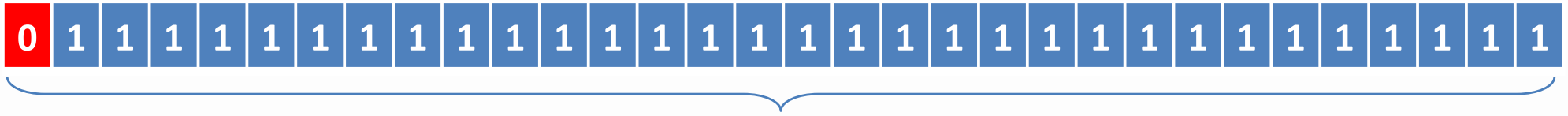



```
#include <iostream>
using namespace std;
int main()
{
        signed int a = 0x7FFFFFFF;
        a = a + 1;
        cout << dec << a << endl;
        return 0;
}
```



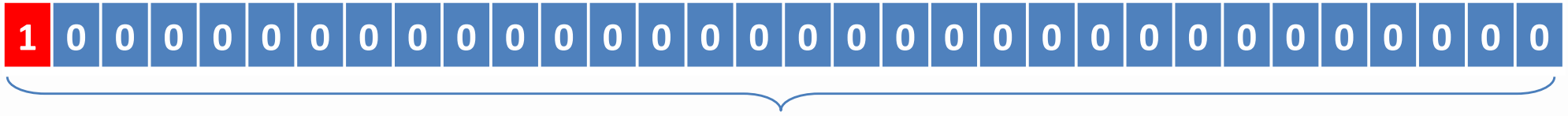
Max & Min

signed int i;



32bit

2147483648



32bit

1

- **当最高位是1其他位为0（-0）时，最高位既表示负号，也表示整数最高位1.（取反加一看一看！）**

= -2147483648



Max & Min

signed int;



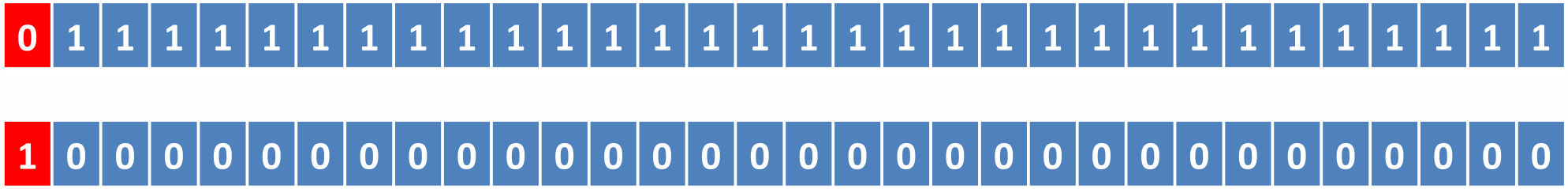


signed int i; 范围是 -2^31 ~ 2^31 - 1



Max & Min

signed int;



signed int i; 范围是 -2^31 ~ 2^31 - 1

unsigned int i;



unsigned int i; 范围是 0 ~ 2^32 - 1

整型数据的范围

- VC中每种类型所占内存空间和表示的范围：

short [int]	2	−32 768～32 767
signed short [int]	2	−32 768～32 767
unsigned short [int]	2	0～65 535
int	4	−2 147 483 648～2 147 483 647
signed int	4	−2 147 483 648～2 147 483 647
unsigned int	4	0～4 294 967 295
long [int]	4	−2 147 483 648～2 147 483 647
signed long [int]	4	−2 147 483 648～2 147 483 647
unsigned long [int]	4	0～4 294 967 295

- 总结：**其实，就用 int 就行了！**



开始其他数据类型

C/C++ 程序中的数据类型


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)



浮点型

浮点型 = 实型　　Visual C++环境👇

浮点型	长度	有效位(精度)	范围
float	32bit	7位	$-3.4*10^{38}$～$3.4*10^{38}$
double	64bit	15位	$-1.7*10^{308}$～$1.7*10^{308}$
long double	64bit	15位	$-1.7*10^{308}$～$1.7*10^{308}$

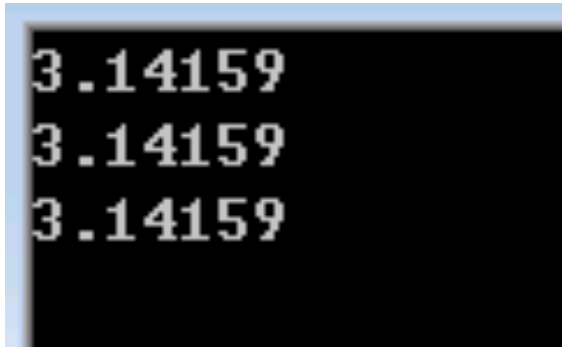
有效位：从第一个非零数字起，直到末尾数字止的所有数字的数量

感受浮点型

```cpp
#include <iostream>
using namespace std;
int main() {
    float a =
        3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79;

    double b =
        3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79;

    long double c =
        3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79;
    cout << a << endl;
    cout << b << endl;
    cout << c << endl;
    return 0;
}
```



感受浮点型

```
3.14159274101257320000000000000000000000
00
3.14159265358979310000000000000000000000
00
3.14159265358979310000000000000000000000
00
```

```
0679;
    cout.precision(100);
    cout << a << endl;
    cout << b << endl;
    cout << c << endl;
    return 0;
}
```

cout.precision 设置输出保留的有效数字。

感受浮点型

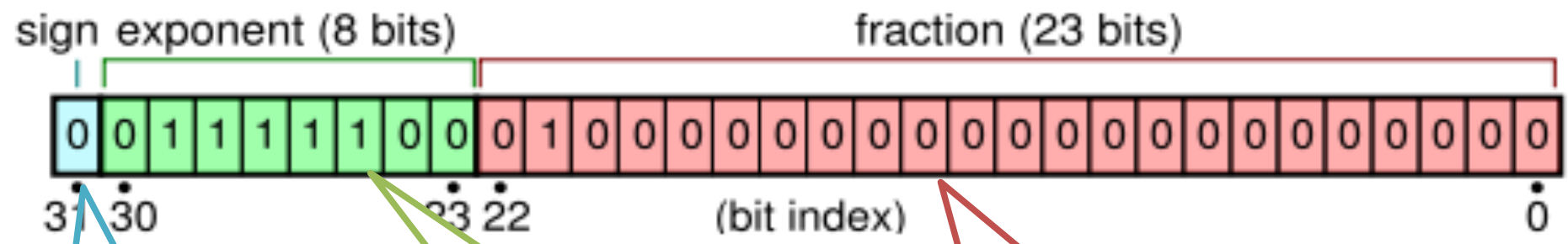
```
#include <iomanip>
#include <iostream>
using namespace std;
int main() {
    float a =
        3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706
    long double c =
        3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706
79;
    cout << setprecision(100) << a << endl;
    cout << b << endl;
    cout << c << endl;
    return 0;
}
```

```
3.1415927410125732000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
3.1415926535897931000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
3.1415926535897931000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
```

setprecision 设置输出保留的有效数字。

需要引入 <iomanip> 库。

浮点数的表示



1位符号位

8位指数位
(含1位符号位)

23位二进制小数位
(默认为X.XXXX)

$$+314.16 = +3.1416 \times (10)^{2}$$

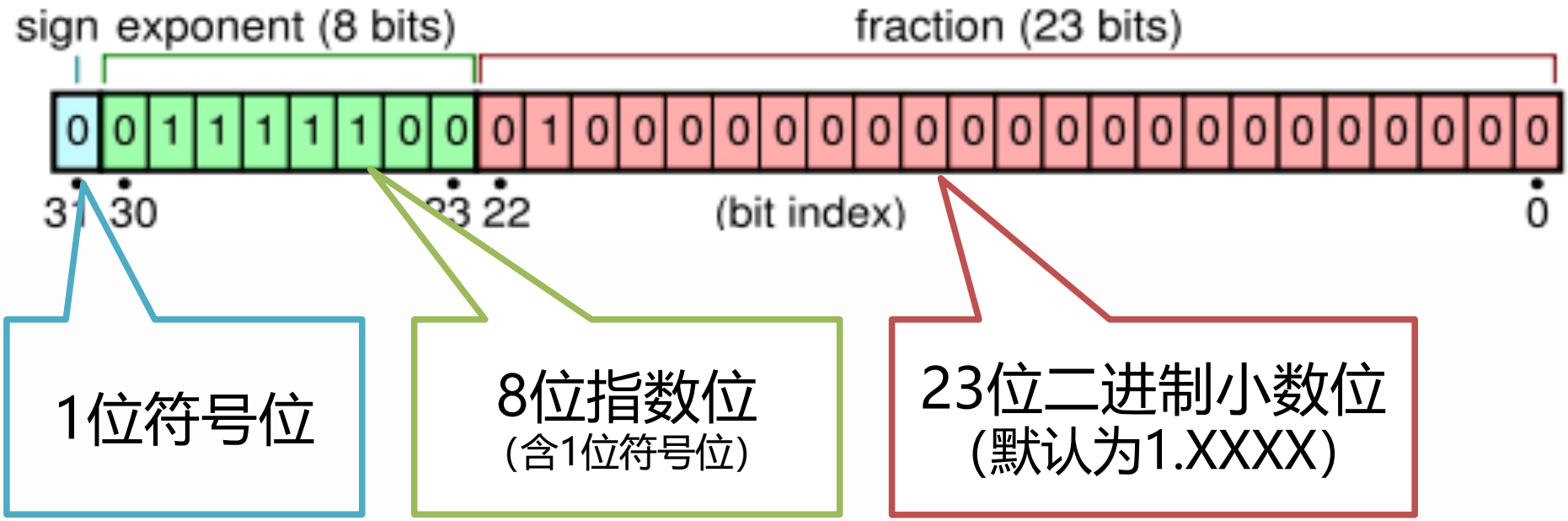
数值精度

数值范围

$$\log_{10}(2^{23}) \approx 6.92369$$

$$\log_{10}(2^{127}) \approx 38.23$$

浮点数的表示



$$\log_{10}(2^{127}) \approx 38.23 \quad \log_{10}(2^{23}) \approx 6.92369$$

- **IEEE Standard for Floating-Point Arithmetic (IEEE 754)**
  - http://grouper.ieee.org/groups/754/
- **ISO/IEC/IEEE 60559:2011 - Information technology - Microprocessor Systems - Floating-Point arithmetic**
  - http://www.iso.org/iso/iso_catalogue/catalogue_tc/catalogue_detail.htm?csnumber=57469

使用须知

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        float a, b;
        a = 123456.789e5;
        b = a + 20;
        cout << setprecision(20) << b << endl;
        return 0;
}
```

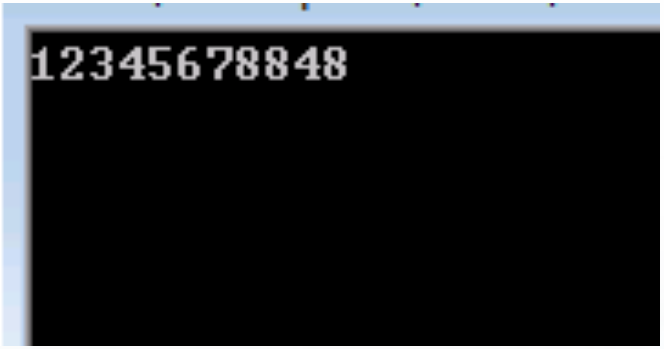
避免将一个很大的数与一个很小的数直接相加或相减，否则就会“丢失”小的数。

支持科学计数法

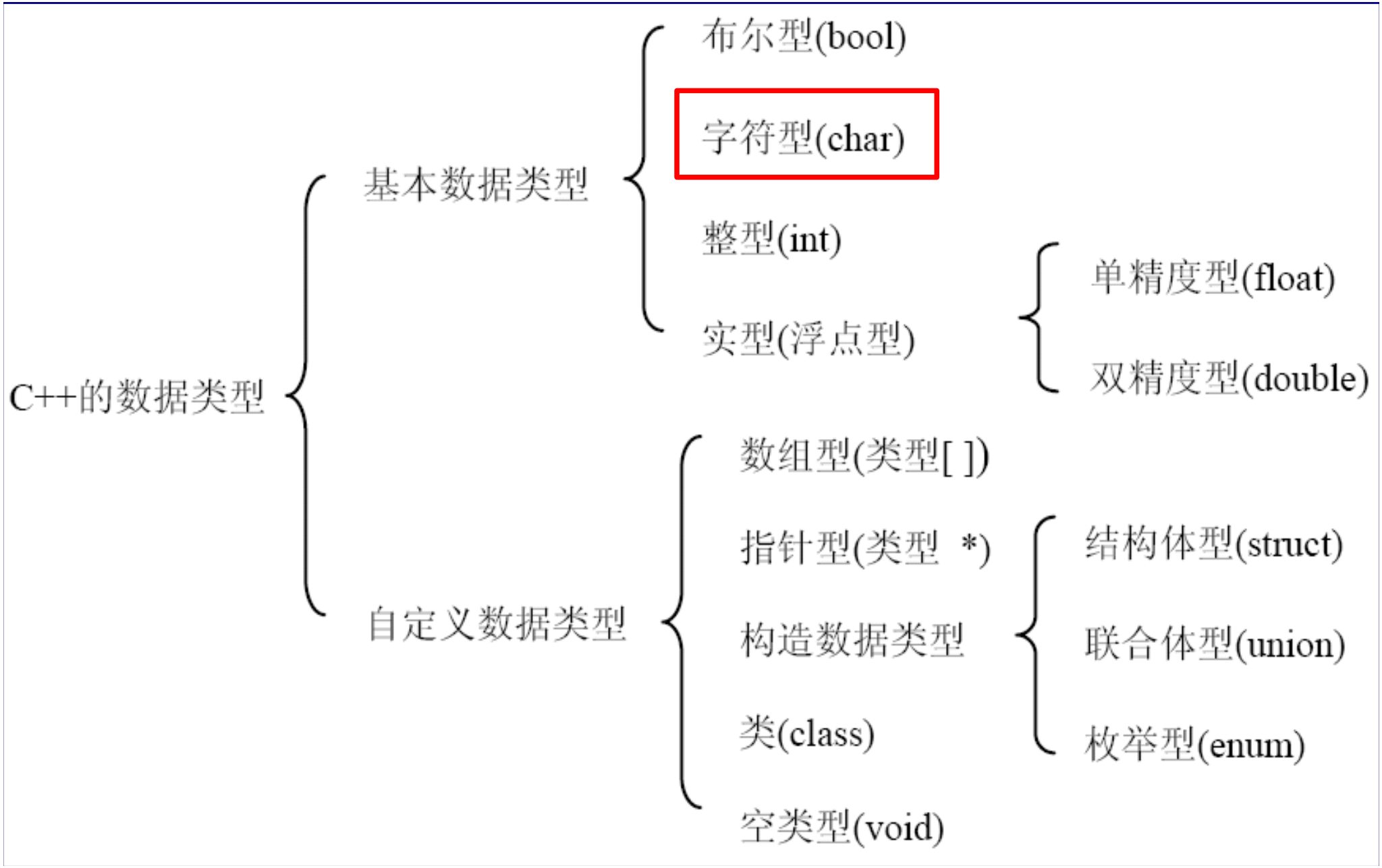
等于 123456.789×10^5

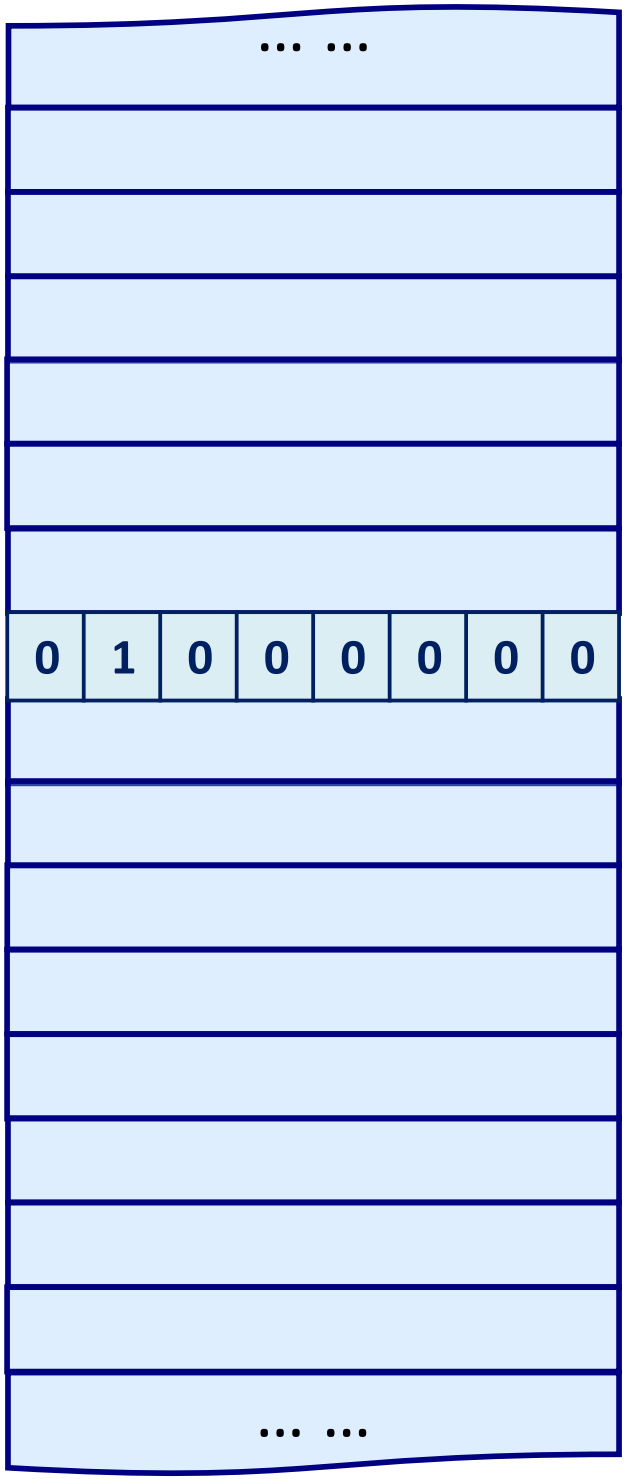
等于 $1.23456789 \times 10^{10}$

等于 12345678900 + 20



C/C++ 程序中的数据类型


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)



字符型数据

- 一个字符型占一个字节；
  - 其值可以是任何“可以出现在C/C++语言中的英文字符”；
  - 最多可以表示

    $2^8 = 256$个字符

- 例如

  char a = ‘@’；

  @ ⟷ 64 ⟷ 01000000

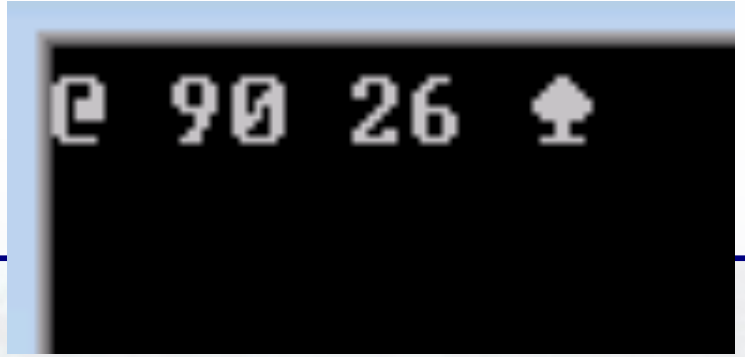
ASCII character Codes

0	<NUL>	32	<SPC>	64	@	96	`	128	Ä	160	†	192	¿	224	‡
1	<SOH>	33	!	65	A	97	a	129	Å	161	°	193	¡	225	·
2	<STX>	34	"	66	B	98	b	130	Ç	162	¢	194	¬	226	‚
3	<ETX>	35	#	67	C	99	c	131	É	163	£	195	√	227	„
4	<EOT>	36	$	68	D	100	d	132	Ñ	164	§	196	ƒ	228	‰
5	<ENQ>	37	%	69	E	101	e	133	Ö	165	•	197	≈	229	Â
6	<ACK>	38	&	70	F	102	f	134	Ü	166	¶	198	Δ	230	Ê
7	<BEL>	39	'	71	G	103	g	135	á	167	ß	199	«	231	Á
8	<BS>	40	(	72	H	104	h	136	à	168	®	200	»	232	Ë
9	<TAB>	41	)	73	I	105	i	137	â	169	©	201	…	233	È
10	<LF>	42	*	74	J	106	j	138	ä	170	™	202		234	Í
11	<VT>	43	+	75	K	107	k	139	ã	171	´	203	À	235	Î
12	<FF>	44	,	76	L	108	l	140	å	172	¨	204	Ã	236	Ï
13	<CR>	45	-	77	M	109	m	141	ç	173	≠	205	Õ	237	Ì
14	<SO>	46	.	78	N	110	n	142	é	174	Æ	206	Œ	238	Ó
15	<SI>	47	/	79	O	111	o	143	è	175	Ø	207	œ	239	Ô
16	<DLE>	48	0	80	P	112	p	144	ê	176	∞	208	–	240	
17	<DC1>	49	1	81	Q	113	q	145	ë	177	±	209	—	241	Ò
18	<DC2>	50	2	82	R	114	r	146	í	178	≤	210	“	242	Ú
19	<DC3>	51	3	83	S	115	s	147	ì	179	≥	211	”	243	Û
20	<DC4>	52	4	84	T	116	t	148	î	180	¥	212	‘	244	Ù
21	<NAK>	53	5	85	U	117	u	149	ï	181	µ	213	’	245	ı
22	<SYN	54	6	86	V	118	v	150	ñ	182	∂	214	÷	246	ˆ
23	<ETB>	55	7	87	W	119	w	151	ó	183	∑	215	◊	247	˜
24	<CAN>	56	8	88	X	120	x	152	ò	184	∏	216	ÿ	248	¯
25	<EM>	57	9	89	Y	121	y	153	ô	185	π	217	Ÿ	249	˘
26	<SUB>	58	:	90	Z	122	z	154	ö	186	∫	218	/	250	˙
27	<ESC>	59	;	91	[	123	{	155	õ	187	ª	219	€	251	°
28	<FS>	60	<	92	\	124	|	156	ú	188	º	220	‹	252	¸
29	<GS>	61	=	93	]	125	}	157	ù	189	Ω	221	›	253	˝
30	<RS>	62	>	94	^	126	~	158	û	190	æ	222	fi	254	˛
31	<US>	63	?	95	_	127	<DEL>	159	ü	191	ø	223	fl	255	ˇ

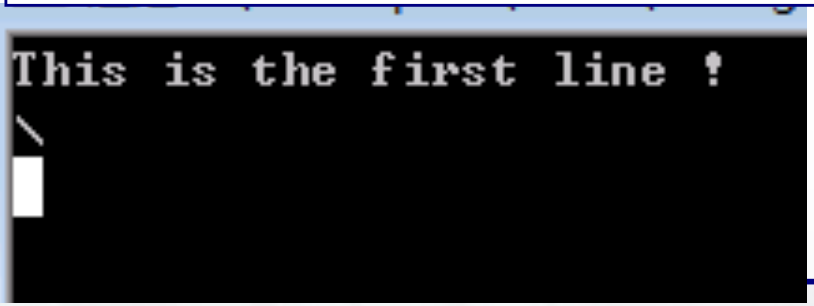
字符型 使用须知 1

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        char a = 64;
        int b = 'Z';
        int c = b - a;
        char d = 6 + 256;
        cout << a << ' ' << b << ' ' << c << ' ' << d << endl;
        return 0;
}
```

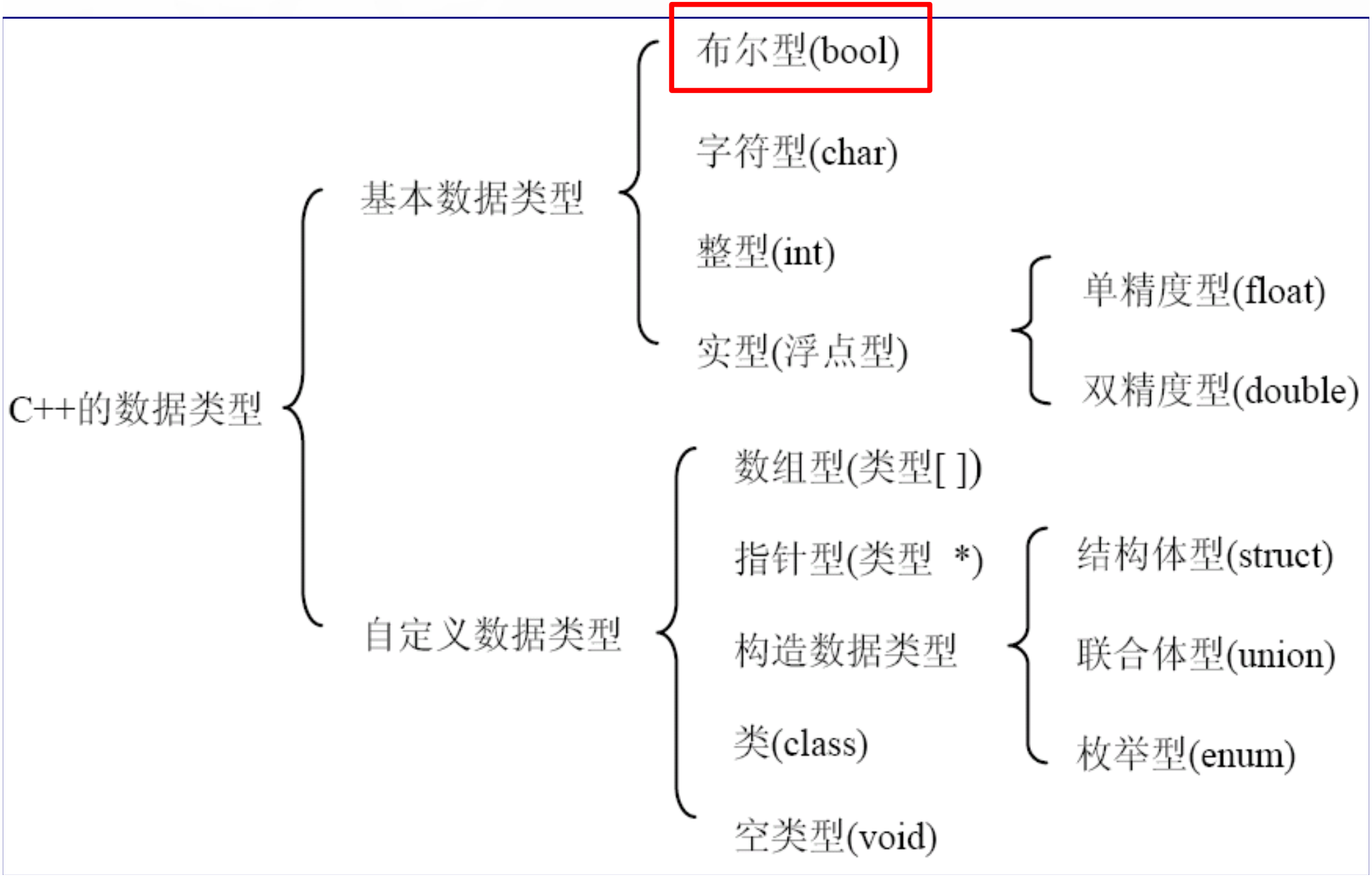
- 由于存储类型和整型相同
  - 可以与整型数据相互赋值
  - 可以和整数一样进行运算

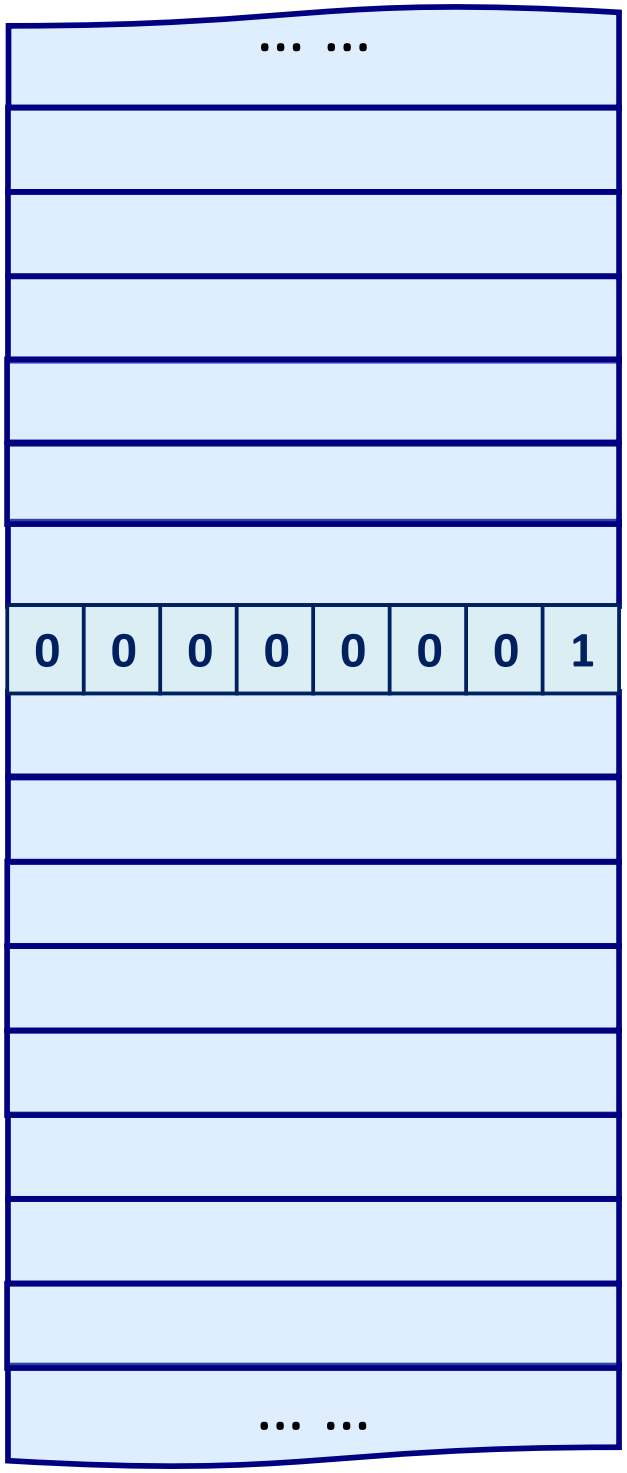


转义字符	描　　述
\a	响铃(audible bell)
\b	退格(backspace)
\f	换页(formfeed)
\n	换行(newline)
\r	回车(carriage return)
\t	水平制表(horizontal tab)
\v	垂直制表(vertical tab)
\\	反斜线(backslash)
\'	单引号(single quote)
\"	双引号(double quote)
\DDD	八进制数 DDD 对应的字符(octal number)
\xHH	十六进制数 HH 对应的字符(hexadecimal number)



C/C++ 程序中的数据类型


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)

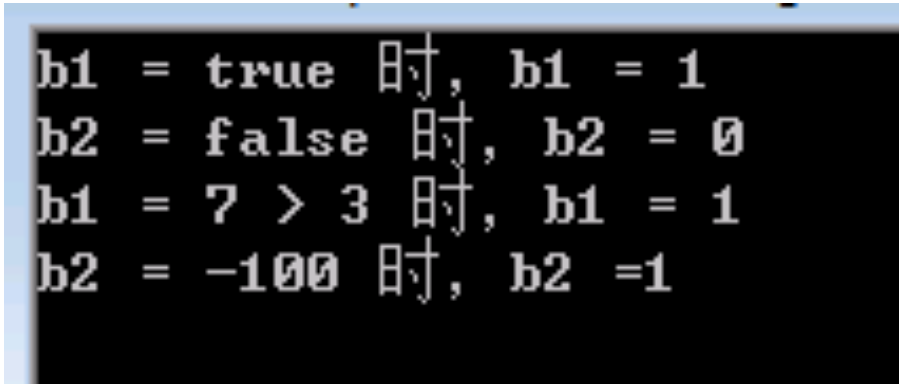


布尔型

- 用于存储“真”和“假”的变量
  - 一般占一个字节 (标准未给定)
- 其值只能为 true 或 false
  - true 在内存中用 1 存储
  - false 在内存中用 0 存储
- 赋给布尔型变量的值
  - true 或 false
  - 关系表达式（如 7 > 3 ）
  - 整型变量
    - 非 0 值为 true
    - 0 值为 false

布尔型

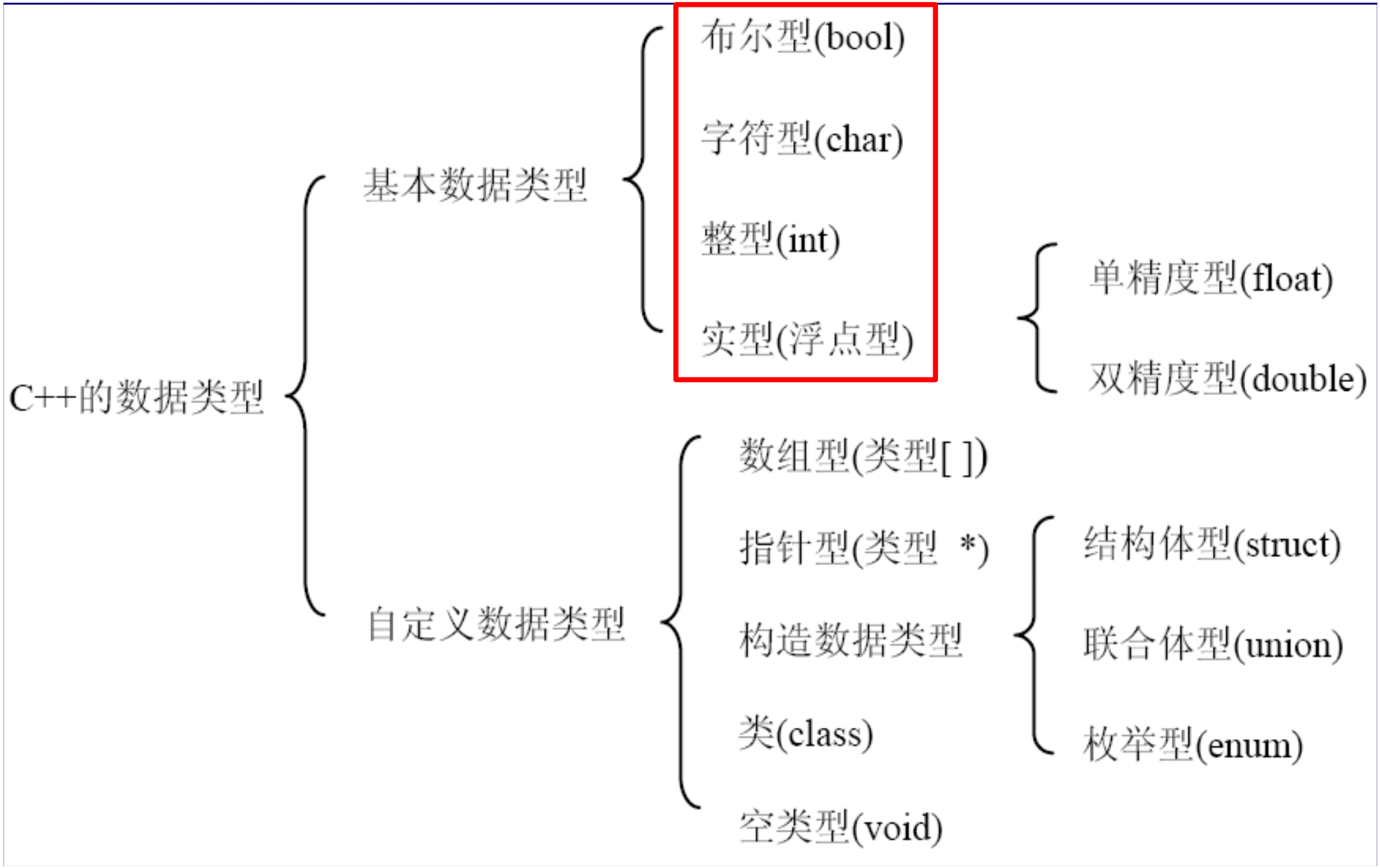
```
#include <iostream>
using namespace std;
int main()
{
    bool b1 = true, b2 = false;
    cout << "b1 = true 时，  b1 = " << b1 << endl;
    cout << "b2 = false 时，  b2 = " << b2 << endl;
    b1 = 7 > 3;
    b2 = -100;
    cout << "b1 = 7 > 3 时，  b1 = " << b1 << endl;
    cout << "b2 = -100 时，  b2 =" << b2 << endl;
    return 0;
}
```



C/C++算术类型

类型名称	含义	一般的字节数	一般的取值范围	
			[signed]	unsigned
bool	布尔类型	(1)	true 或 false	
char	字符类型	1	ASCII字符	
short	短整型	2	$-2^{15} \sim 2^{15}-1$	$0 \sim 2^{16}$
int	整型	4	$-2^{31} \sim 2^{31}-1$	$0 \sim 2^{32}$
long	长整型	4	$-2^{31} \sim 2^{31}-1$	$0 \sim 2^{32}$
long long	扩展长整型	8	$-2^{63} \sim 2^{63}-1$	$0 \sim 2^{64}$
float	单精度浮点型	4	6~7位有效数字	
double	双精度浮点型	8	15~16位有效数字	
long double	扩展精度浮点型	>=8	≥ 15位有效数字	

C/C++ 程序中的数据类型


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)



所有“数”都是有类型的

——常量 也一样！

C/C++中的常量

- 常量：编译时确定的值。在程序运行过程中，其值保持不变
  - 字面量
    - -1，0，123，4.6，-1.23；
  - 符号常量 constexpr
    - 用一个标识符代表一个常量的，称为符号常量

```
#include <iostream>
using namespace std;
int main()
{
  constexpr float PI = 3.14159f;
  float  r, area;
  cin >> r;
  area = r * r * PI;
  cout << "Area = " << area;
  return 0;
}
```



C/C++中的字面量

- 字面量：直接表示为一个具体的数或字符串的值。
- 字面量及由字面量组成的表达式的值在程序运行**前**，即编译的时候，已经被惟一**确定**。
  - ◆整数类型字面量：

    ```
    -1; 3+5; 123; 'X';
    ```

  - ◆浮点类型字面量：

    ```
    7.5; 6e-2; -3.14;
    ```

  - ◆“字符串字面量”：

    ```
    "Hello";
    ```



字面量有类型吗?

- **整型字面量的后缀**

```
10000L;      //长整型常量
-0x88abL;    //长整型十六进制常量
10000U;      //无符号整型常量
07777LU;     //无符号长整型八进制常量
```

- **浮点型字面量的后缀**

```
3.1415F;     //单精度浮点型常量
3.1415L;     //长双精度浮点型常量
```

- **说明:**
  - **整型字面量默认为 `int` 型，浮点型字面量默认为 `double` 型;**
  - **`U`, `L`, `F`均可以小写。**



希望

了解但不陷入细节

等用到时再细究它

谢 谢!

预告：数组型


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
类(class)
枚举型(enum)
空类型(void)

由相同类型的变量可以组成——数组

```
char a[10] = { 'a', 'b','c', 'd', 'e','f','g','h','i','j'};
```

a	b	c	d	e	f	g	h	i	j
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
1	2	3	4	5	6	7	8	9	10

```
int a[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

预告：结构体


C++的数据类型
基本数据类型
布尔型(bool)
字符型(char)
整型(int)
实型(浮点型)
单精度型(float)
双精度型(double)
自定义数据类型
数组型(类型[ ])
指针型(类型 *)
构造数据类型
结构体型(struct)
联合体型(union)
枚举型(enum)
类(class)
空类型(void)



问题

- 可不可以把多个变量“组合”在一起?

```
{
    int     id;
    char    name[20];
    char    sex;
    int     age;
    float   score;
    char    addr[30];
}
```



定义自己的变量：
把不同的变量组合在一起
——结构体



结构体

- 声明一个名为“学生”的结构体

```
struct student \\结构体的名字为“student”;
 {
        int     id;          \\声明学号为int型;
        char   name[20];  \\声明姓名为字符数组;
        char   sex;         \\声明性别为字符型;
        int     age;        \\声明年龄为整型;
        float   score;    \\声明成绩为实型;
        char   addr[30];  \\声明地址为字符数组
};          \\注意大括号后的“;”
```



定义结构体类型的变量

- 定义结构体变量

  student　　student1, student2;

  （结构体类型名）　（结构体变量名）；

  - 对比：

    int a;　（ struct student 相当于 int ）

    float a; (struct student 相当于 float)

结构体变量的引用

- 引用结构体变量中成员的方式为

  结构体变量名.成员名

  - 如： student1.id = 10010;

    student1.birthday.month = 10；

- 不能将一个结构体变量作为一个整体进行输入和输出
  - 不正确的引用：cout<<student1;　　cin>>student1;
- 只能对结构体变量中的各个成员分别进行输入和输出
  - 正确的引用： cin>>student1.id;　cout<<student1.id;

结构体示例

```
int main( )
{
        int day = 7;
        date Birthday;
        Birthday.day = 25;
        Birthday.month = 12;
        Birthday.year = 0;
        cout<<day<<endl;
        cout<<Birthday.day<<endl;
        return 0;
}
```

```
struct  date
{
   int    month;
    int    day;
    int    year;
};
```